Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи № 6
«Автоматизований контроль захищеності у конвеєрі CI/CD»
з дисципліни «Безпека програмного забезпечення»

Варіант 12

Виконав: здобувач групи ІП-33
Ломакін Максим

Перевірив:
Курченко Олег Анастасійович

Київ — 2026

1 МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи – навчитися вбудовувати автоматизовані перевірки захищеності в конвеєр неперервної інтеграції та доставки (CI/CD) і налаштовувати критерій проходження (quality gate) – умову, за якою складання зупиняється в разі виявлення дефекту безпеки.

Завдання: налаштувати конвеєр GitHub Actions зі стадіями статичного аналізу, аналізу складу ПЗ та сканування секретів; розробити власне правило Semgrep; сформувати SBOM у форматі CycloneDX; реалізувати в `gate.py` критерій проходження варіанта; застосувати конвеєр до навмисно вразливого застосунку з лабораторної роботи 5; оцінити частку хибних спрацювань.

Варіант 12. Інструмент SAST – Semgrep із власним правилом; критерій проходження – блокувати складання за наявності секрету в діффі. Вивід команди `python3 -m common.variants 12 --lab 6`:

```
Варіант 12

  LAB6
    sast: Semgrep + власне правило
    gate: блокувати за наявності секрету в діффі
```

Docker на робочій машині не встановлено, тому інструменти встановлено у віртуальне середовище Python, а застосунок ЛР5 не запускався: усі стадії аналізують його статично. Конвеєр у GitHub Actions не запускався; його кроки відтворено локально тими самими командами. У звіті результати позначено як отримані локально, а про GitHub Actions сказано лише те, що описано у файлі конвеєра.

2 ТЕОРЕТИЧНІ ВІДОМОСТІ

Вартість усунення дефекту зростає приблизно на порядок з кожним етапом життєвого циклу: $C_n = C_0 \cdot k^n$, $k \approx 10$. Тому практики захищеної розробки NIST SSDF [1] вимагають, щоб перевірки були частиною складання, а не разовим аудитом [5]. Звідси перше рішення роботи: усі перевірки запускаються подією `push` і `pull_request`, а рішення «пропустити чи заблокувати» ухвалює програма, а не людина.

Статичний аналіз (SAST) зіставляє вихідний код із шаблонами небезпечних конструкцій без його виконання [2]. Semgrep описує шаблон декларативно мовою цільового коду, тому власне правило – це кілька рядків YAML: `pattern` задає синтаксичну форму, метазмінні (`$APP`, `$PATH`) – довільні підвирази, а `metavariable-regex` і `pattern-not-regex` звужують збіг [7]. Обмеження методу випливає з його

природи: правило бачить форму коду, а не його зміст. Дефекти, що полягають у відсутності перевірки (IDOR, CSRF), не мають характерної синтаксичної форми, і виявити їх можна лише там, де ознаку вдається сформулювати через структуру – наприклад, «маршрут /admin без звернення до ролі». Кожна знахідка класифікується за переліком CWE [6] і потребує ручної оцінки: частина збігів є хибнопозитивними.

Критерій проходження – функція від звітів стадій: $g = (m_1 > t_1) \vee \ldots \vee (m_k > t_k)$. Варіант 12 задає ознаку «секрет у діффі», тож виміряними величинами є кількість секретів, які знайшов gitleaks [8] у комітах діффу, і кількість збігів власного правила Semgrep про жорстко закодований секрет на рядках, доданих діффом; обидва пороги дорівнюють нулю. Прив'язка до діффу, а не до всього коду, означає, що критерій оцінює саме зміну: він не карає за успадковані дефекти, але й не помічає їх. Блокування реалізується ненульовим кодом завершення процесу, який система CI трактує як провал кроку.

Секрет, що потрапив в історію Git, лишається в ній після вилучення файла, тому перевіряти треба коміти, а не лише робоче дерево. З цієї ж причини звіт сканера не повинен містити значення секрету: gitleaks запускається з ключем `--redact`.

Аналіз складу ПЗ (SCA) зіставляє версії залежностей із базами вразливостей, а специфікація SBOM у форматі CycloneDX [3] фіксує перелік компонентів складання і забезпечує простежуваність, якої вимагають рівні SLSA [4]. Оцінка CVSS [9] дає числову шкалу 0–10, придатну для порога блокування.

3 ОПИС РОЗРОБЛЕНОГО РІШЕННЯ

3.1 Структура конвеєра

Конвеєр `security` (рисунок 3.1) запускається на кожен `push` і `pull_request` та складається з трьох незалежних стадій. Стадія `sast` формує звіти Bandit і Semgrep, обчислює діфф події, шукає секрети в його комітах і викликає `gate.py`. Стадія `sca` перевіряє залежності та будує SBOM, стадія `secrets` запускає офіційну дію gitleaks над комітами події. Токен робочого процесу має лише право `contents: read`.

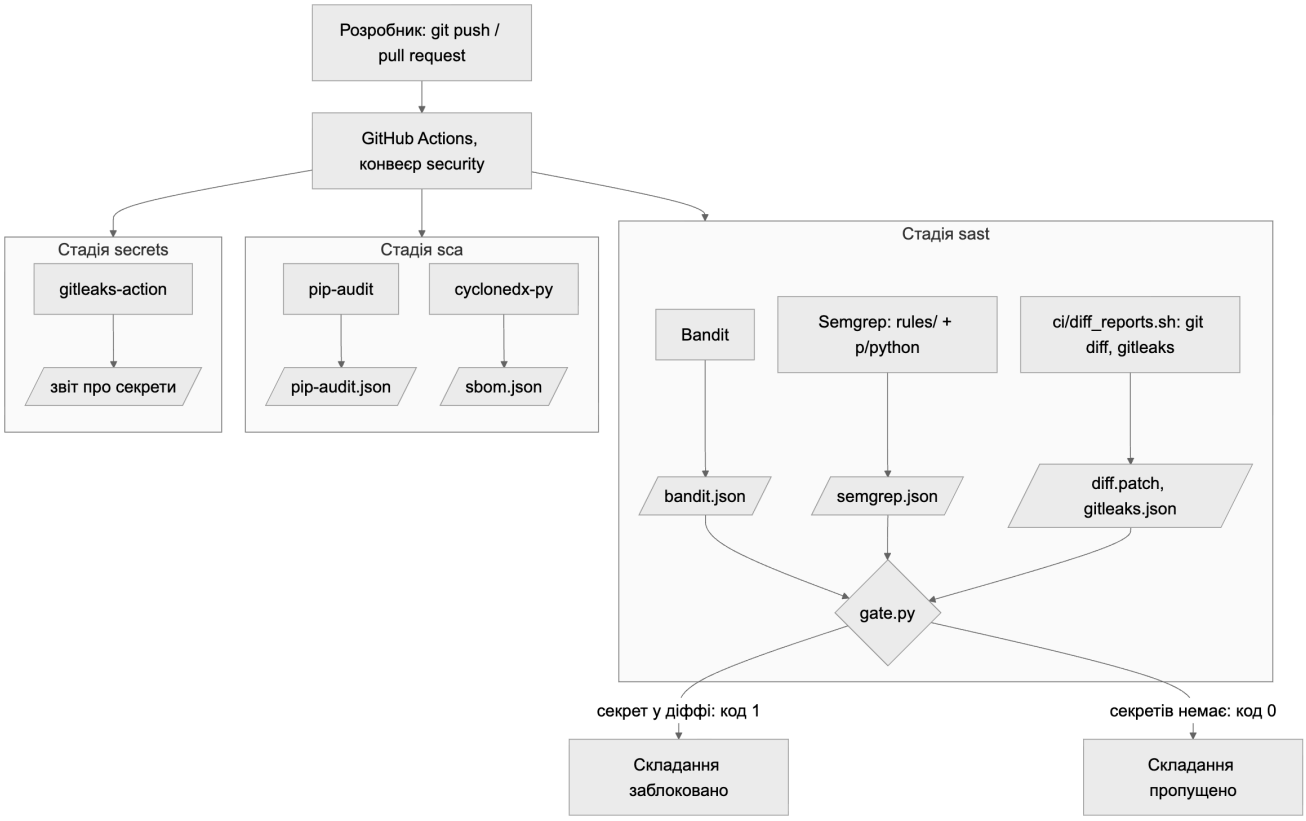


Рисунок 3.1 – Структура конвеєра контролю захищеності

Версії програмних засобів наведено в таблиці 3.1. Python-інструменти встановлено з `requirements.txt` каркаса; gitleaks узято з офіційного випуску, контрольну суму архіву перевірено. Semgrep 1.86.0 під Python 3.12 потребує пакета `setuptools` (імпортує `pkg_resources`), тому його додано до команди встановлення в конвеєрі.

Таблиця 3.1 – Програмні засоби

Засіб	Версія	Призначення
Python	3.12.0	запуск інструментів і `gate.py`
Bandit	1.7.10	статичний аналіз коду Python
Semgrep	1.86.0	статичний аналіз, власні правила і набір `p/python`
pip-audit	2.7.3	пошук вразливих залежностей
cyclonedx-bom	4.6.1	формування SBOM у форматі CycloneDX
gitleaks	8.21.2	сканування секретів у файлах і комітах
Git	2.50.1	історія комітів, обчислення діффу

Каркас із репозиторію курсу [11] доповнено такими файлами (таблиця 3.2).

Таблиця 3.2 – Файли роботи

Файл	Вміст
`rules/insecure_patterns.yml`	до трьох прикладів каркаса додано три власні правила Semgrep
`gate.py`	критерій проходження «секрет у діффі»

Файл	Вміст
ci/diff_reports.sh	формує diff.patch і gitleaks.json для діффу події
.github/workflows/security.yml	конвеєр: додано gitleaks і діфф у стадію sast, збереження звітів
fix/secret_key.patch	виправлення дефекту, на який реагує критерій
demo_states.sh	відтворення чотирьох станів критерію в тимчасовому репозиторії
sca_cvss.py	оцінки CVSS з бази NVD для знахідок pip-audit
tests/test_gate.py	19 тестів

3.2 Власні правила Semgrep

Варіант 12 у лабораторній роботі 5 охоплює відсутність перевірки прав в адміністративному маршруті, XSS, CSRF та IDOR. Для двох із цих класів характерну синтаксичну ознаку сформулювати можна, і для них написано правила; третє правило стосується критерію проходження.

```
- id: my-lomakin-hardcoded-secret-key
  languages: [python]
  severity: ERROR
  message: >-
    Секрет сесії Flask задано рядковим літералом або літералом за
    замовчуванням (CWE-798). Читайте його лише з оточення.
  metadata:
    cwe: "CWE-798: Use of Hard-coded Credentials"
  patterns:
    - pattern-either:
        - pattern: $APP.secret_key = "..."
        - pattern: $APP.secret_key = os.environ.get($NAME, "...")
        - pattern: $APP.secret_key = os.getenv($NAME, "...")
        - pattern: $APP.config["SECRET_KEY"] = "..."
        - pattern: $APP.config["SECRET_KEY"] = os.environ.get($NAME, "...")
        - pattern: $APP.config["SECRET_KEY"] = os.getenv($NAME, "...")

- id: my-lomakin-admin-route-no-authz
  languages: [python]
  severity: ERROR
  message: >-
    Адміністративний маршрут $PATH не перевіряє роль користувача (CWE-862).
  metadata:
    cwe: "CWE-862: Missing Authorization"
  patterns:
    - pattern: |
        @$APP.route("$PATH", ...)
        def $FUNC(...):
            ...
    - metavariable-regex:
        metavariable: $PATH
        regex: ^/admin(/|$)
    - pattern-not-regex: \[['"]role['"]\]

- id: my-lomakin-jinja-safe-filter
  languages: [generic]
  severity: WARNING
  message: >-
    Фільтр |safe вимикає автоекранування Jinja2 — значення потрапляє в HTML
    без кодування (CWE-79).
  metadata:
    cwe: "CWE-79: Cross-site Scripting"
  paths:
```

```
    include: ["*.html"]
  pattern-regex: \{\{[^}]*\|\s*safe\s*\}\}
```

my-lomakin-hardcoded-secret-key, CWE-798. Шаблон описує присвоєння атрибуту `secret_key` або ключу конфігурації `SECRET_KEY` рядкового літерала (`"..."` у Semgrep означає будь-який рядок) – безпосередньо або як значення за замовчуванням у `os.environ.get` чи `os.getenv`. Другий випадок і є дефектом застосунку: якщо змінну оточення не задано, сесії підписуються відомим рядком, і cookie сесії можна підробити. Саме цей збіг враховує критерій проходження.

my-lomakin-admin-route-no-authz, CWE-862. Шаблон збігається з функцією, що має декоратор `route`; `metavariable-regex` залишає лише шляхи, які починаються з `/admin`, а `pattern-not-regex` вилучає функції, у тексті яких є звернення до поля `["role"]`. Правило формулює відсутність перевірки через структуру коду.

my-lomakin-jinja-safe-filter, CWE-79. Правило мови `generic` шукає регулярним виразом вираз Jinja2 з фільтром `|safe` у файлах `*.html`. Фільтр вимикає автоматичне екранування, тому дані користувача потрапляють у сторінку як розмітка – це причина і відображеного, і збереженого XSS у застосунку.

3.3 Критерій проходження

Критерій блокує складання, якщо істинна хоча б одна з двох умов: gitleaks знайшов секрет у комітах діффу або правило `my-lomakin-hardcoded-secret-key` збіглося на рядку, який додає діфф. Потрібні звіти готує сценарій `ci/diff_reports.sh`, який викликається в конвеєрі перед `gate.py`:

```
#!/usr/bin/env bash
# Готує для gate.py звіти про діфф BASE..HEAD: diff.patch і gitleaks.json.
# Якщо BASE не задано або такого коміту немає, діффом вважається вся історія.
set -u
base="${1:-}"
if [ -n "$base" ] && git cat-file -e "${base}^{commit}" 2>/dev/null; then
  range="${base}..HEAD"
else
  base="$(git hash-object -t tree /dev/null)"
  range="HEAD"
fi
echo "Діфф: ${range}"
git diff -U0 --no-color "$base" HEAD > diff.patch
# --redact: значення секрету до звіту не потрапляє.
gitleaks detect --source . --log-opts="$range" --redact --no-banner \
  --report-format json --report-path gitleaks.json || true
```

Базовий коміт конвеєр бере з події: для `pull_request` – коміт цільової гілки, для `push` – попередній стан гілки. Якщо такого коміту немає (перший `push` гілки, ручний запуск), діффом вважається вся історія. Номери доданих рядків функція `added_lines` бере із заголовків фрагментів `@@ -a,b +c,d @@` діффу без контексту, а `rule_secrets` залишає лише ті збіги правила, що перетинаються з ними:

```
def gitleaks_secrets(report: list) -> list[str]:
    """Знахідки gitleaks у вигляді «правило файл:рядок» без значення секрету."""
    return [f"{r.get('RuleID')} {r.get('File')}:{r.get('StartLine')}"
            for r in report]


def added_lines(diff: str) -> dict[str, set[int]]:
    """Номери рядків, доданих діффом (формат git diff -U0), за файлами."""
    added: dict[str, set[int]] = {}
    path = None
    for line in diff.splitlines():
        if line.startswith("+++ "):
            name = line[4:].strip()
            path = None if name == "/dev/null" else name.removeprefix("b/")
        elif path and (m := HUNK.match(line)):
            start, count = int(m.group(1)), int(m.group(2) or "1")
            added.setdefault(path, set()).update(range(start, start + count))
    return added


def rule_secrets(semgrep: dict, added: dict[str, set[int]],
                 rule: str = SECRET_RULE) -> list[str]:
    """Збіги власного правила Semgrep, що припадають на додані рядки."""
    hits = []
    for r in semgrep.get("results", []):
        if not r.get("check_id", "").endswith(rule):
            continue
        lines = range(r["start"]["line"], r["end"]["line"] + 1)
        if added.get(r["path"], set()).intersection(lines):
            hits.append(f"{rule} {r['path']}:{r['start']['line']}")
    return hits
```

У функції `main()` новий критерій стоїть після наявної перевірки Bandit. Перевірку HIGH збережено, але вона діє, лише якщо задано змінну `MAX_HIGH`: варіант 12 її не передбачає, а застосунок ЛР5 має три знахідки HIGH, тож із порогом 0 складання лишалося б заблокованим незалежно від секретів. Фрагмент `main()`, що реалізує критерій варіанта:

```
    gitleaks, semgrep = load("gitleaks.json"), load("semgrep.json")
    diff = read_text("diff.patch")
    missing = [name for name, value in (("gitleaks.json", gitleaks),
               ("semgrep.json", semgrep), ("diff.patch", diff)) if value is None]
    if missing or not isinstance(gitleaks, list) or not isinstance(semgrep, dict):
        print("::error::Quality gate: звіт для критерію секретів відсутній або"
              f" пошкоджений ({', '.join(missing) or 'невірний формат'})"
              " — складання заблоковано")
        return 1
    leaks = gitleaks_secrets(gitleaks)
    hardcoded = rule_secrets(semgrep, added_lines(diff))
    print(f"Секретів у діффі за gitleaks: {len(leaks)}")
    print(f"Збігів правила {SECRET_RULE} на доданих рядках: {len(hardcoded)}")
    if leaks or hardcoded:
        for item in leaks + hardcoded:
            print(f"::error::Quality gate: секрет у діффі — {item}")
        return 1
    print("Quality gate: пройдено")
    return 0
```

Відсутній або пошкоджений звіт трактується як збій сканера, і складання блокується: сканери в конвеєрі запускаються з `|| true`, тож інакше їхнє падіння

виглядало б як відсутність знахідок. У повідомлення потрапляють лише правило, файл і рядок – без значення секрету.

3.4 Файл конвеєра

Кроки стадії `sast` після отримання коду й налаштування Python (у кроці `checkout` задано `fetch-depth: 0`, бо для діффу потрібна історія):

```
- name: Встановлення інструментів
  # setuptools: semgrep 1.86.0 імпортує pkg_resources, якого немає в 3.12
  run: pip install bandit==1.7.10 semgrep==1.86.0 "setuptools<81"
- name: Встановлення gitleaks
  run: |
    file="gitleaks_${GITLEAKS_VERSION}_linux_x64.tar.gz"
    url="https://github.com/gitleaks/gitleaks/releases/download"
    curl -fsSL -o "$RUNNER_TEMP/$file" "$url/v${GITLEAKS_VERSION}/$file"
    echo "${GITLEAKS_SHA256}  $RUNNER_TEMP/$file" | sha256sum -c -
    mkdir -p "$RUNNER_TEMP/bin"
    tar -xzf "$RUNNER_TEMP/$file" -C "$RUNNER_TEMP/bin" gitleaks
    echo "$RUNNER_TEMP/bin" >> "$GITHUB_PATH"
# Ціль SAST — каталог target/ із копією застосунку ЛР5 (див. методичку, крок 7).
- name: Bandit
  run: bandit -r target -f json -o bandit.json || true
- name: Semgrep (власні правила + реєстр)
  run: semgrep --config rules/ --config "p/python" --json --output semgrep.json target
|| true
# Звіти для критерію варіанта: діфф події та секрети в його комітах.
- name: Діфф і секрети в діффі
  env:
    BASE: ${{ github.event.pull_request.base.sha || github.event.before }}
  run: ci/diff_reports.sh "$BASE"
- name: Quality gate
  run: python gate.py
```

У стадії `sca` додано машинний звіт pip-audit і виправлено команду формування SBOM (підрозділ 4.3); у стадії `secrets` дії gitleaks передано токен робочого процесу. Обидві стадії зберігають звіти як артефакти запуску. Файл розібрано як YAML і перевірено тестом; у GitHub Actions він ще не виконувався.

4 РЕЗУЛЬТАТИ

4.1 Статичний аналіз застосунку ЛР5

Застосунок ЛР5 скопійовано в каталог `target/` і проаналізовано командами з файла конвеєра. Bandit завершився з кодом 1 (знахідки є), Semgrep – з кодом 0; з'явилися `bandit.json` (12 знахідок, серед них B324, B602 і B301) та `semgrep.json` (21 знахідка, помилок аналізу немає). Перевірка власних правил командою з кроку 3 (`semgrep --config rules/ --json target`) друкує перелік `check_id`:

```
['rules.my-lomakin-hardcoded-secret-key',
 'rules.python-md5-for-password',
 'rules.my-lomakin-admin-route-no-authz',
 'rules.my-lomakin-admin-route-no-authz',
 'rules.python-subprocess-shell-true',
 'rules.python-md5-for-password',
 'rules.my-lomakin-jinja-safe-filter',
 'rules.my-lomakin-jinja-safe-filter']
```

Усі знахідки Semgrep з оцінкою наведено в таблиці 4.1; виділено 5 збігів власних правил. Стовпець «ЛР5» містить номер позначеної вразливості застосунку, якщо знахідка їй відповідає; стовпець CWE – клас, який оголошує саме правило (для знахідки 2 це CWE-704, хоча за змістом вона є SQL-ін'єкцією, CWE-89).

Таблиця 4.1 – Знахідки Semgrep на застосунку ЛР5

№	Правило	Рівень	CWE	Файл:рядок	Маршрут	ЛР5	Оцінка
1	**my-lomakin-hardcoded-secret-key**	**ERROR**	**CWE-798**	**app.py:34**	**– (секрет сесії)**	**1**	**істинне**
2	tainted-sql-string	ERROR	CWE-704	app.py:75	/login	2	істинне
3	insecure-hash-algorithm-md5	WARNING	CWE-327	app.py:101	/login	4	істинне
4	python-md5-for-password	WARNING	CWE-916	app.py:101	/login	4	істинне
5	md5-used-as-password	WARNING	CWE-327	app.py:101	/login	4	істинне
6	**my-lomakin-admin-route-no-authz**	**ERROR**	**CWE-862**	**app.py:213**	**/admin/users**	**12**	**істинне**
7	**my-lomakin-admin-route-no-authz**	**ERROR**	**CWE-862**	**app.py:221**	**/admin/ping**	**–**	**істинне**
8	subprocess-injection	ERROR	CWE-78	app.py:225	/admin/ping	13	істинне
9	python-subprocess-shell-true	ERROR	CWE-78	app.py:225	/admin/ping	13	істинне
10	dangerous-subprocess-use	ERROR	CWE-78	app.py:226	/admin/ping	13	істинне
11	subprocess-shell-true	ERROR	CWE-78	app.py:226	/admin/ping	13	істинне
12	directly-returned-format-string	WARNING	CWE-79	app.py:228	/admin/ping	–	істинне
13	raw-html-format	WARNING	CWE-79	app.py:228	/admin/ping	–	істинне
14	raw-html-format	WARNING	CWE-79	app.py:228	/admin/ping	–	істинне
15	avoid_app_run_with_bad_host	WARNING	CWE-668	app.py:237	– (запуск)	–	хибне
16	avoid_app_run_with_bad_host	WARNING	CWE-668	internal.py:31	– (запуск)	–	хибне
17	insecure-hash-algorithm-md5	WARNING	CWE-327	seed.py:69	– (наповнення БД)	4	істинне
18	python-md5-for-password	WARNING	CWE-916	seed.py:69	– (наповнення БД)	4	істинне
19	md5-used-as-password	WARNING	CWE-327	seed.py:69	– (наповнення БД)	4	істинне

№	Правило	Рівень	CWE	Файл:рядок	Маршрут	ЛР5	Оцінка
20	**my-lomakin-jinja-safe-filter**	**WARNING**	**CWE-79**	**templates/document.html:9**	**/document/<id>**	**7**	**істинне**
21	**my-lomakin-jinja-safe-filter**	**WARNING**	**CWE-79**	**templates/search.html:8**	**/search**	**6**	**істинне**

Знахідки Bandit наведено в таблиці 4.2. Рівень HIGH мають 3 з них.

Таблиця 4.2 – Знахідки Bandit на застосунку ЛР5

№	Перевірка	Рівень	CWE	Файл:рядок	Маршрут	ЛР5	Оцінка
1	B403	LOW	CWE-502	app.py:22	/prefs (імпорт)	–	хибне
2	B404	LOW	CWE-78	app.py:24	/admin/ping (імпорт)	–	хибне
3	B108	MEDIUM	CWE-377	app.py:29	– (шлях до БД)	–	хибне
4	B608	MEDIUM	CWE-89	app.py:76	/login	2	істинне
5	B324	HIGH	CWE-327	app.py:101	/login	4	істинне
6	B310	MEDIUM	CWE-22	app.py:195	/fetch	10	істинне
7	B301	MEDIUM	CWE-502	app.py:207	/prefs	11	істинне
8	B602	HIGH	CWE-78	app.py:226	/admin/ping	13	істинне
9	B104	MEDIUM	CWE-605	app.py:237	– (запуск)	–	хибне
10	B104	MEDIUM	CWE-605	internal.py:31	– (запуск)	–	хибне
11	B324	HIGH	CWE-327	seed.py:69	– (наповнення БД)	4	істинне
12	B108	MEDIUM	CWE-377	seed.py:85	– (шлях до БД)	–	хибне

Таблиця 4.3 зіставляє знахідки з переліком 13 позначених вразливостей застосунку.

Таблиця 4.3 – Покриття позначених вразливостей застосунку ЛР5

№	CWE	Маршрут	Клас	Semgrep	Bandit
1	CWE-798	секрет сесії	жорстко закодований секрет	власне правило	–
2	CWE-89	/login	SQL-ін'єкція	так	так
3	CWE-209	/login	розкриття помилки СУБД	–	–
4	CWE-916	зберігання пароля	слабке хешування без солі	так	так
5	CWE-639	/document/<id>	IDOR	–	–
6	CWE-79	/search	відображений XSS	власне правило	–
7	CWE-79	/comment/<id>	збережений XSS	власне правило	–
8	CWE-352	/profile	CSRF	–	–
9	CWE-22	/download	обхід каталогів	–	–
10	CWE-918	/fetch	SSRF	–	так
11	CWE-502	/prefs	небезпечна десеріалізація	–	так
12	CWE-862	/admin/users	відсутня перевірка прав	власне правило	–
13	CWE-78	/admin/ping	ін'єкція команди ОС	так	так

4.2 Критерій проходження: стани блокування і пропуску

Критерій залежить від діффу, тому його перевірено в тимчасовому git-репозиторії, який створює `demo_states.sh`: чотири послідовні коміти, після кожного – ті самі команди, що й у стадії `sast`. Виправлення для стану 2:

```
--- a/target/app.py
+++ b/target/app.py
@@ -30,8 +30,8 @@
 UPLOAD_DIR = "/app/storage"

 app = Flask(__name__)
-# ВРАЗЛИВІСТЬ 1 (CWE-798): жорстко закодований передбачуваний секрет сесії.
-app.secret_key = os.environ.get("VULNAPP_SECRET", "dev-secret")
+# Секрет сесії читається лише з оточення; без нього застосунок не стартує.
+app.secret_key = os.environ["VULNAPP_SECRET"]


 # ── Доступ до бази даних ──────────────────────────────────────────
```

Для стану 3 створено файл із ключем-пустушкою (дзеркальна шістнадцяткова послідовність, не є ключем жодного сервісу), позначений коментарем «ТЕСТОВИЙ КЛЮЧ». Вивід сценарію:

```
== Стан 1: застосунок ЛР5 додано до репозиторію ==
коміт: a874807
Діфф: HEAD
Знахідок Bandit рівня HIGH: 3
Секретів у діффі за gitleaks: 0
Збігів правила my-lomakin-hardcoded-secret-key на доданих рядках: 1
::error::Quality gate: секрет у діффі — my-lomakin-hardcoded-secret-key
target/app.py:34
код завершення: 1
== Стан 2: секрет сесії читається лише з оточення ==
коміт: 1c09f13
Діфф: a874807..HEAD
Знахідок Bandit рівня HIGH: 3
Секретів у діффі за gitleaks: 0
Збігів правила my-lomakin-hardcoded-secret-key на доданих рядках: 0
Quality gate: пройдено
код завершення: 0
== Стан 3: до коміту потрапив тестовий ключ ==
коміт: 5ba60f3
Діфф: 1c09f13..HEAD
Знахідок Bandit рівня HIGH: 3
Секретів у діффі за gitleaks: 1
Збігів правила my-lomakin-hardcoded-secret-key на доданих рядках: 0
::error::Quality gate: секрет у діффі — generic-api-key target/settings_local.py:2
код завершення: 1
-- gitleaks detect --no-git (робоче дерево) --
код завершення gitleaks: 1
generic-api-key target/settings_local.py 2 LAB6_FAKE_API_KEY =
"0f1e2d3c4b5a69788796a5b4c3d2e1f0"
== Стан 4: тестовий ключ вилучено ==
коміт: 5df1182
Діфф: 5ba60f3..HEAD
Знахідок Bandit рівня HIGH: 3
Секретів у діффі за gitleaks: 0
Збігів правила my-lomakin-hardcoded-secret-key на доданих рядках: 0
Quality gate: пройдено
код завершення: 0
-- gitleaks detect --no-git (робоче дерево) --
код завершення gitleaks: 0
записів у звіті: 0
-- gitleaks detect (уся історія) --
код завершення gitleaks: 1
generic-api-key target/settings_local.py 5ba60f3
```

Таблиця 4.4 – Стани критерію проходження

Стан	Зміна в діффі	gitleaks	Власне правило	Код gate.py	Рішення
1	до репозиторію додано застосунок ЛР5 із секретом сесії за замовчуванням	0	1	1	заблоковано
2	секрет сесії читається лише з оточення	0	0	0	пропущено
3	у коміті є файл із тестовим ключем	1	0	1	заблоковано
4	файл із тестовим ключем вилучено	0	0	0	пропущено

У станах 3 і 4 виконано самоперевірку кроку 5: сканування робочого дерева знаходить ключ (правило `generic-api-key`, поле `Match` містить рядок із ключем), а після вилучення файла звіт порожній. Сканування каталогу роботи тією самою командою з ключем `--no-git` секретів не виявило (код 0, `gitleaks.json` порожній). Запуск `python gate.py` у каталозі без діффу завершується з кодом 1:

```
Знахідок Bandit рівня HIGH: 3
::error::Quality gate: звіт для критерію секретів відсутній або пошкоджений
(diff.patch) — складання заблоковано
```

4.3 Аналіз складу ПЗ та SBOM

Команда `pip-audit -r audit_target.txt` (код завершення 1):

```
Found 5 known vulnerabilities in 2 packages
Name   Version ID              Fix Versions
------ ------- --------------- ------------
pyyaml 5.3.1   PYSEC-2021-142  5.4
jinja2 2.11.3  PYSEC-2026-1473 3.1.3
jinja2 2.11.3  PYSEC-2026-1471 3.1.6
jinja2 2.11.3  PYSEC-2026-1474 3.1.4
jinja2 2.11.3  PYSEC-2026-1475 3.1.5
```

Оцінок CVSS у звіті немає, тому `sca_cvss.py` для кожного запису `pip-audit.json` запитує базу NVD за псевдонімом CVE і бере найбільшу базову оцінку CVSS v3.1 (таблиця 4.5).

Таблиця 4.5 – Вразливі залежності навчального переліку та оцінки CVSS за NVD

Пакет	Версія	Ідентифікатор	CVE	Виправлено	CVSS	≥ 7.0	≥ 4.0
pyyaml	5.3.1	PYSEC-2021-142	CVE-2020-14343	5.4	9.8	так	так
jinja2	2.11.3	PYSEC-2026-1473	CVE-2024-22195	3.1.3	6.1	–	так
jinja2	2.11.3	PYSEC-2026-1471	CVE-2025-27516	3.1.6	8.8	так	так
jinja2	2.11.3	PYSEC-2026-1474	CVE-2024-34064	3.1.4	5.4	–	так
jinja2	2.11.3	PYSEC-2026-1475	CVE-2024-56326	3.1.5	7.8	так	так

Команда з опису роботи `cyclonedx-py requirements -i audit_target.txt -o sbom.json` дала SBOM із компонентами bandit, cyclonedx-bom, pip-audit, semgrep – тобто для `requirements.txt`: у версії 4.6.1 ключ `-i` задає адресу індексу пакетів, а файл вимог є позиційним аргументом. Тому SBOM сформовано командою `cyclonedx-py requirements audit_target.txt -o sbom.json`, і так само

виправлено крок конвеєра. Перевірка документа друкує формат, версію специфікації та кількість компонентів:

```
CycloneDX 1.5 2
Jinja2 2.11.3 pkg:pypi/jinja2@2.11.3
PyYAML 5.3.1 pkg:pypi/pyyaml@5.3.1
```

4.4 Тести

Файл `tests/test_gate.py` містить 19 тестів: розбір діффу, обидві умови блокування, пропуск чистого діффу, відмова за відсутнього чи пошкодженого звіту, спрацювання власних правил на коді застосунку та його зникнення після виправлення, структура файла конвеєра. Запуск: `PYTHONPATH=. pytest tests -v`.

```
============================= test session starts ==============================
collecting ... collected 19 items

tests/test_gate.py::test_bandit_high_counts_only_high PASSED
tests/test_gate.py::test_added_lines_parses_hunks PASSED
tests/test_gate.py::test_added_lines_of_empty_diff PASSED
tests/test_gate.py::test_gitleaks_secrets_do_not_expose_value PASSED
tests/test_gate.py::test_rule_secrets_only_on_added_lines PASSED
tests/test_gate.py::test_gate_passes_on_clean_diff PASSED
tests/test_gate.py::test_gate_blocks_on_gitleaks_finding PASSED
tests/test_gate.py::test_gate_blocks_on_rule_match_in_diff PASSED
tests/test_gate.py::test_gate_ignores_rule_match_outside_diff PASSED
tests/test_gate.py::test_gate_ignores_other_rules_and_high_by_default PASSED
tests/test_gate.py::test_max_high_threshold_is_optional PASSED
tests/test_gate.py::test_gate_fails_closed_without_report[bandit.json] PASSED
tests/test_gate.py::test_gate_fails_closed_without_report[gitleaks.json] PASSED
tests/test_gate.py::test_gate_fails_closed_without_report[semgrep.json] PASSED
tests/test_gate.py::test_gate_fails_closed_without_report[diff.patch] PASSED
tests/test_gate.py::test_gate_fails_closed_on_broken_report PASSED
tests/test_gate.py::test_own_rules_match_real_code PASSED
tests/test_gate.py::test_fix_removes_secret_rule_match PASSED
tests/test_gate.py::test_workflow_structure PASSED

============================== 19 passed in 2.17s ==============================
```

5 АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

1. Кількість знахідок SAST, їх розподіл і покриття 13 вразливостей. Semgrep (власні правила і набір `p/python`) дав 21 знахідку у 13 місцях коду: 8 рівня ERROR і 13 рівня WARNING. За походженням: 13 – правила реєстру, 3 – приклади каркаса, 5 – власні правила. За класами, які оголошують самі правила: CWE-79 – 5, CWE-78 – 4, CWE-327 – 4, CWE-668 – 2, CWE-862 – 2, CWE-916 – 2, CWE-704 – 1, CWE-798 – 1. Правило `tainted-sql-string` позначає SQL-ін'єкцію як CWE-704, хоча за змістом це CWE-89. Із 13 позначених вразливостей Semgrep виявив 7: № 1, 2, 4, 6, 7, 12, 13; без власних правил – лише 3 (№ 2, 4, 13). Bandit дав 12 знахідок (HIGH – 3, MEDIUM – 7, LOW – 2) і додав № 10 та № 11, яких Semgrep не побачив; разом два інструменти покривають 9 із 13. Жоден не виявив № 3, 5, 8, 9. IDOR (№ 5) і CSRF (№ 8) – це відсутність перевірки власника та токена: запит `SELECT … WHERE id = ?` і звичайний

`UPDATE` синтаксично бездоганні, а дефект полягає в тому, чого в коді немає. Розкриття помилки СУБД (№ 3) – звичайна передача рядка в шаблон, яку не відрізнити від легітимного повідомлення. Обхід каталогів (№ 9) має характерну форму `os.path.join` із параметром запиту, але ні набір `p/python`, ні Bandit такого правила не містять – його можна додати власним правилом.

2. Частка хибнопозитивних спрацювань. Після ручної звірки кожної знахідки з кодом хибними визнано 2 із 21 знахідок Semgrep (9,5 %) і 6 із 12 знахідок Bandit (50 %). Приклад: `avoid_app_run_with_bad_host` на рядку `app.run(host="0.0.0.0", …)` (CWE-668). У контейнері прив'язка до всіх інтерфейсів потрібна, інакше застосунок недоступний з мережі стенда, а назовні порт публікується лише на 127.0.0.1 – це задає `docker-compose.yml`, якого правило не бачить. У Bandit хибними є ще й інформаційні B403 і B404 (сам імпорт `pickle` чи `subprocess` дефектом не є, небезпечні виклики позначено окремо як B301 і B602) та B108 (шлях `/tmp/vulnapp.db` у контейнері з одним користувачем). Водночас істинні знахідки Semgrep дублюються: 21 записів відповідають 13 місцям, наприклад рядок із `hashlib.md5` позначають три правила одразу. Три знахідки CWE-79 у рядку 228 – реальний дефект поза переліком 13: вивід `ping`, що містить параметр `host`, повертається як HTML без кодування; динамічно це не перевірялося.

3. Власні правила Semgrep. Усі три правила спрацювали на реальному коді застосунку й дали 5 збігів: `my-lomakin-hardcoded-secret-key` – 1 (рядок 34, № 1), `my-lomakin-admin-route-no-authz` – 2 (/admin/users, № 12, і /admin/ping), `my-lomakin-jinja-safe-filter` – 2 (шаблони search.html і document.html, № 6 і № 7). Хибних серед них немає: маршрут /admin/ping теж не перевіряє роль, хоча в переліку його позначено лише як ін'єкцію команди. Правила евристичні. Правило про роль не побачить маршрут, який звертається до `user["role"]`, але порівнює його неправильно, і дасть хибне спрацювання, якщо перевірку винесено в декоратор. Правило про `|safe` позначить і навмисне виведення перевіреного HTML. Збіг правила про секрет сесії враховує критерій проходження.

4. Вразливі залежності та поріг CVSS. pip-audit визнав вразливими обидві залежності з `audit_target.txt` і знайшов 5 вразливостей: одну в PyYAML 5.3.1 і чотири в Jinja2 2.11.3. Оцінок CVSS pip-audit 2.7.3 не виводить – лише ідентифікатор і версію з виправленням, тому оцінки отримано з бази NVD [10] за псевдонімом CVE: CVE-2020-14343 – 9.8, CVE-2024-22195 – 6.1, CVE-2025-27516 – 8.8, CVE-2024-34064 – 5.4, CVE-2024-56326 – 7.8. За порога 7.0 критерій CVSS-варіанта блокував би складання через 3 вразливості, за порога 4.0 – через усі 5: додаються дві знахідки середнього рівня (6.1 і 5.4). Саме рішення не змінюється – складання блокується в

обох випадках, бо максимальна оцінка 9.8. Зміна стала б помітною після оновлення PyYAML до 5.4 і Jinja2 до 3.1.6 – у цих версіях виправлено всі п'ять знайдених вразливостей. Якщо ж оновити Jinja2 лише до 3.1.3, залишаться оцінки 8.8, 7.8 і 5.4. Коментар у каркасі стверджує, що вразливості Jinja2 мають CVSS < 7.0, але за даними NVD дві з них оцінено у 8.8 і 7.8; у записах GitHub ті самі вразливості мають рівень Moderate, тож результат залежить від джерела оцінки.

5. Стани, у яких критерій блокує і пропускає складання. Критерій блокує складання у двох станах: коли діфф додає рядок, на якому спрацьовує власне правило про секрет сесії (стан 1, код завершення 1), і коли gitleaks знаходить секрет у комітах діффу (стан 3, код 1). Він пропускає складання, коли діфф прибирає значення за замовчуванням (стан 2, код 0) і коли діфф лише вилучає файл із ключем (стан 4, код 0). Крім того, без будь-якого зі звітів критерій завершується з кодом 1 – відмова в разі збою сканера. Три знахідки Bandit рівня HIGH на рішення не впливають: поріг `MAX_HIGH` варіантом не задано. Стан 4 показує межу критерію: діфф чистий, але сканування всієї історії й далі знаходить ключ у коміті 5ba60f3. Тому справжній секрет після такого випадку треба відкликати, а не лише видалити. Друга межа – успадковані дефекти: якби секрет сесії вже був у репозиторії до ввімкнення конвеєра, діфф його не містив би, і критерій мовчав би, хоча повний звіт Semgrep цю знахідку містить.

6. Чому перевірки запускаються на кожен коміт. За формулою $C_n = C_o \cdot k^n$ дефект, знайдений на етапі кодування, на порядок дешевший за знайдений під час тестування і на два порядки – за знайдений в експлуатації. Перевірка на кожен коміт утримує n на найменшому можливому значенні: дефект виявляється в тій самій зміні, що його внесла, поки автор пам'ятає контекст, а зміна мала. Для секретів аргумент ще сильніший: у стані 3 ключ зупинено до злиття, і достатньо не зливати гілку; після публікації історії ключ уже скомпрометовано, і вартістю стає його відкликання та перевірка журналів доступу. Одноразовий аудит такого не дає: між аудитами кожен коміт може додати новий дефект, а критерій, прив'язаний до діффу, взагалі має сенс лише за умови, що перевіряється кожен діфф – пропущена зміна вже ніколи не потрапить у поле зору.

ВИСНОВКИ

1. Підготовлено конвеєр GitHub Actions із трьома стадіями й локально відтворено всі його кроки: Semgrep дав 21 знахідку, Bandit – 12, pip-audit – 5 вразливостей у двох залежностях, SBOM CycloneDX 1.5 містить 2 компоненти. У GitHub Actions конвеєр не запускався.

2. Додано три власні правила Semgrep, які дали 5 збігів без хибних. З ними Semgrep виявляє 7 із 13 позначених вразливостей застосунку ЛР5 замість 3; разом із Bandit – 9. Вразливості № 3, 5, 8, 9 статичний аналіз не виявив.
3. Частка хибнопозитивних спрацювань становить 9,5 % для Semgrep і 50 % для Bandit; 21 знахідка Semgrep припадає на 13 місць коду.
4. Критерій «секрет у діффі» реалізовано в `gate.py`: у чотирьох станах тимчасового репозиторію коди завершення дорівнюють 1, 0, 1, 0. Критерій відмовляє, якщо бракує будь-якого зі звітів, і не зберігає значення секрету у звіті.
5. Виявлено дві розбіжності з описом роботи: команда `cyclonedx-py requirements -i audit_target.txt` будує SBOM для `requirements.txt`, а не для навчального переліку, і pip-audit не подає оцінок CVSS. За даними NVD поріг 7.0 перевищують 3 з 5 вразливостей, зокрема дві в Jinja2.
6. Написано 19 тестів для критерію, власних правил і структури файла конвеєра; усі проходять.

ПЕРЕЛІК ПОСИЛАНЬ

1. NIST SP 800-218. Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities / M. Souppaya, K. Scarfone, D. Dodson. – Gaithersburg : NIST, 2022.
2. Chess B. Secure Programming with Static Analysis / B. Chess, J. West. – Boston : Addison-Wesley, 2007. – 624 p.
3. CycloneDX Specification [Електронний ресурс] / OWASP Foundation. – Режим доступу: https://cyclonedx.org/
4. SLSA – Supply-chain Levels for Software Artifacts [Електронний ресурс] / OpenSSF. – Режим доступу: https://slsa.dev/
5. Kim G. The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations / G. Kim, J. Humble, P. Debois, J. Willis. – 2nd ed. – Portland : IT Revolution Press, 2021. – 480 p.
6. Common Weakness Enumeration (CWE) [Електронний ресурс] / MITRE. – Режим доступу: https://cwe.mitre.org/
7. Semgrep Documentation: Rule syntax [Електронний ресурс] / Semgrep, Inc. – Режим доступу: https://semgrep.dev/docs/writing-rules/rule-syntax
8. Gitleaks [Електронний ресурс]. – Режим доступу: https://github.com/gitleaks/gitleaks

9. Common Vulnerability Scoring System v3.1: Specification Document [Електронний ресурс] / FIRST. – Режим доступу: https://www.first.org/cvss/v3.1/specification-document
10. National Vulnerability Database [Електронний ресурс] / NIST. – Режим доступу: https://nvd.nist.gov/
11. Соколовський В. В. Навчально-методичний комплекс дисципліни «Безпека програмного забезпечення»: код лабораторних робіт [Електронний ресурс]. – Режим доступу: https://github.com/sokolusha/software-security-course

ДОДАТОК А. ПРОГРАМНИЙ КОД

Повний програмний код роботи розміщено в репозиторії: https://github.com/kqjqxb/Software-Security, коміт 123d71c. Файли роботи – у каталозі `lab6/lab06_cicd`: `gate.py`, `rules/insecure_patterns.yml`, `ci/diff_reports.sh`, `.github/workflows/security.yml`, `fix/secret_key.patch`, `demo_states.sh`, `sca_cvss.py`, `tests/test_gate.py`; артефакти – `bandit.json`, `semgrep.json`, `pip-audit.json`, `cvss.json`, `sbom.json`, `gitleaks.json`.